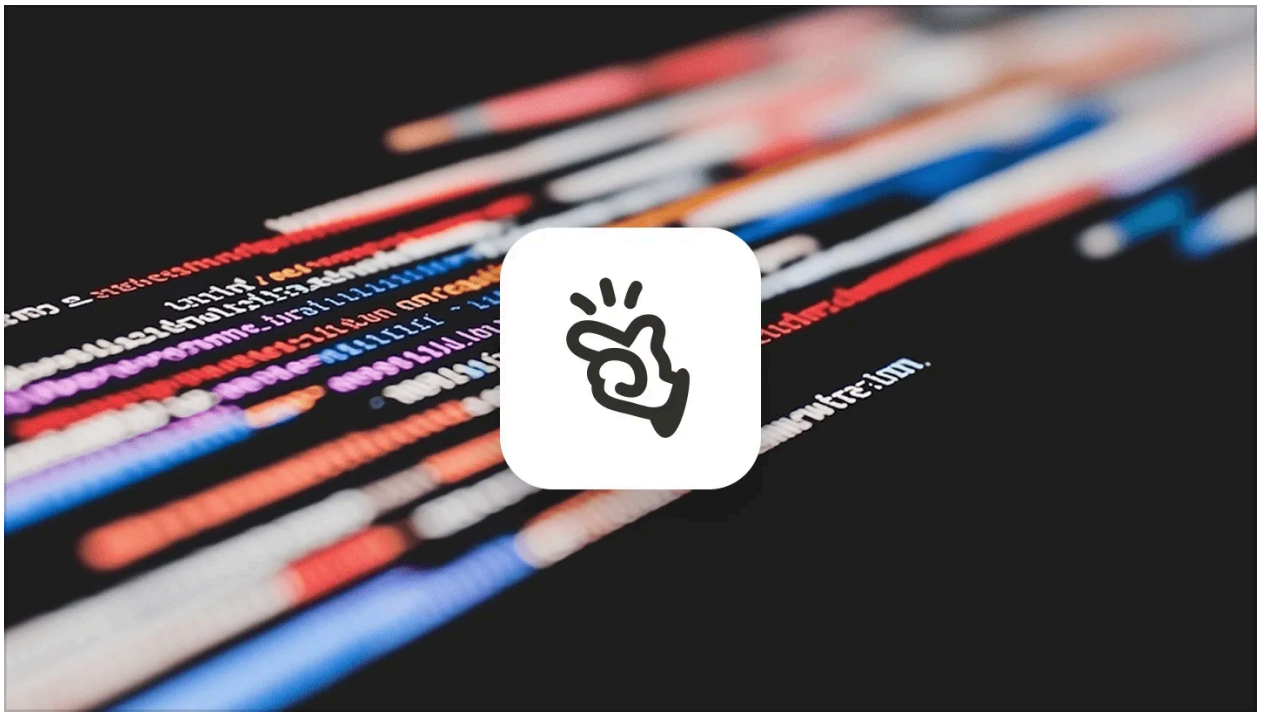
AI代理的上下文工程：构建Manus的经验教训

manus.im/zh-cn/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus

Manus 现已成为 Meta 的一部分

产品·星期五, 7月 18



2025/7/18 --Yichao 'Peak' Ji

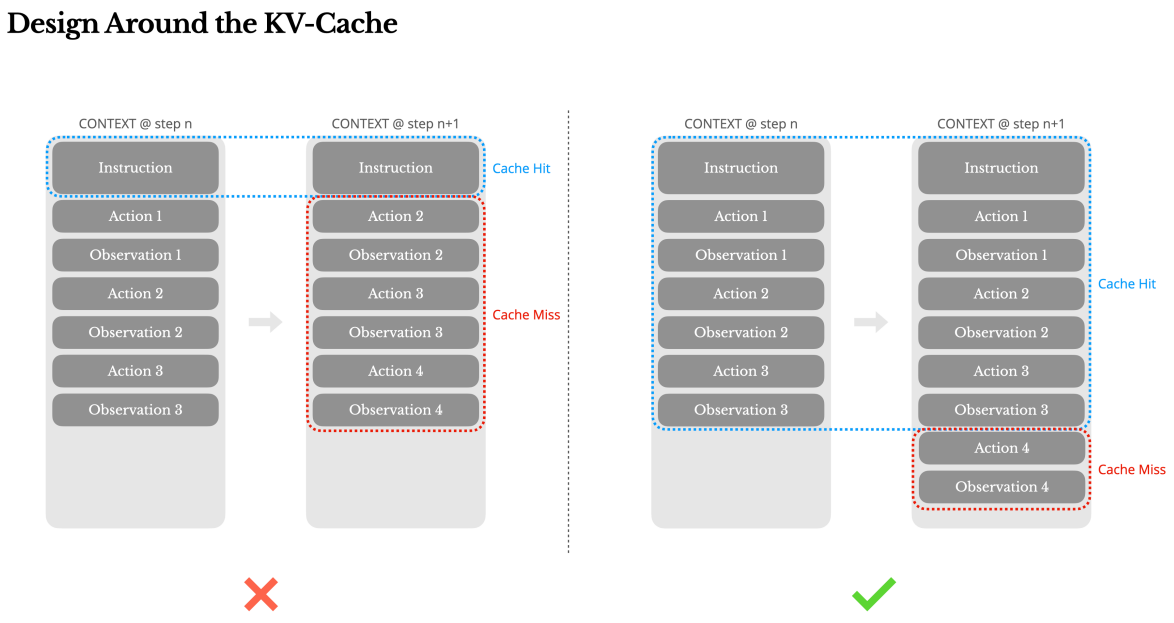
在Manus项目的最初阶段，我和我的团队面临一个关键决策：我们是应该使用开源基础模型训练一个端到端的智能体模型，还是基于前沿模型的上下文学习能力构建一个智能体？在我的NLP生涯的第一个十年里，我们没有这种选择的奢侈。在遥远的BERT时代（是的，已经过去七年了），模型必须先进行微调——和评估——才能迁移到新任务。这个过程通常每次迭代需要数周时间，尽管与今天的LLM相比，这些模型非常小。对于快速发展的应用，特别是在产品市场匹配(PMF)之前，这种缓慢的反馈循环是一个致命缺陷。这是我上一个创业公司的惨痛教训，当时我从头开始训练模型用于开放信息提取和语义搜索。然后GPT-3和Flan-T5出现了，我的内部模型一夜之间变得无关紧要。具有讽刺意味的是，这些相同的模型标志着上下文学习的开始——以及一条全新的前进道路。这个来之不易的教训使选择变得明确：Manus将押注于上下文工程。这使我们能够在几小时而非几周内交付改进，并使我们的产品与底层模型保持正交：如果模型进步是上涨的潮水，我们希望Manus成为那条船，而不是固定在海床上的柱子。
尽管如此，上下文工程证明绝非易事。这是一门实验科学——我们已经重建了我们的代理框架四次，每次都是在发现了更好的塑造上下文的方式之后。我们亲切地将这种手动架构搜索、提示调整和经验猜测的过程称为"随机研究生下降"。这并不优雅，但它有效。

这篇文章分享了我们通过自己的"SGD"所达到的局部最优解。如果你正在构建自己的AI代理，我希望这些原则能帮助你更快地收敛。

围绕KV缓存进行设计

如果我必须选择一个指标，我认为 KV-cache命中率 是生产阶段AI代理最重要的单一指标。它直接影响延迟和成本。为了理解原因，让我们看看典型代理是如何运作的：在接收用户输入后，代理通过一系列工具使用链来完成任务。在每次迭代中，模型根据当前上下文从预定义的动作空间中选择一个动作。然后在环境中执行该动作（例如，Manus的虚拟机沙盒）以产生观察结果。动作和观察结果被附加到上下文中，形成下一次迭代的输入。这个循环持续进行，直到任务完成。正如你所想象的，随着每一步的推进，上下文不断增长，而输出——通常是结构化的函数调用——保持相对简短。这使得代理（agents）相比聊天机器人的预填充和解码比例高度倾斜。例如在Manus中，平均输入与输出的token比例约为100:1。

幸运的是，具有相同前缀的上下文可以利用KV缓存，这大大减少了首个token的生成时间(TTFT)和推理成本——无论你是使用自托管模型还是调用推理API。我们说的不是小幅度的节省：例如使用Claude Sonnet时，缓存的输入token成本为0.30美元/百万token，而未缓存的成本为3美元/百万token——相差10倍。



从上下文工程的角度，提高KV缓存命中率涉及几个关键实践：
1.保持你的提示前缀稳定。 由于LLM的自回归特性，即使是单个标记的差异也会使该标记之后的缓存失效。一个常见的错误是在系统提示的开头包含时间戳——尤其是精确到秒的时间戳。虽然这让模型能告诉你当前时间，但也会降低你的缓存命中率。
2.使你的上下文只追加。 避免修改之前的操作或观察。确保你的序列化是确定性的。许多编程语言和库在序列化JSON对象时不保证键顺序的稳定性，这可能会悄无声息地破坏缓存。

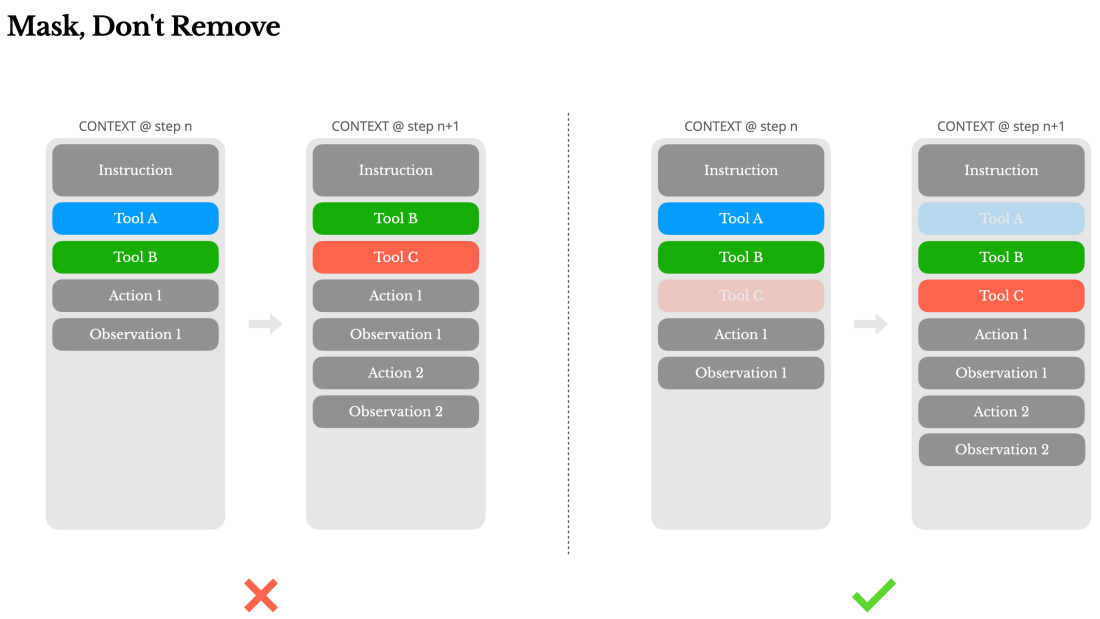
3.在需要时明确标记缓存断点。 某些模型提供商或推理框架不支持自动增量前缀缓存，而是需要在上下文中手动插入缓存断点。在分配这些断点时，要考虑潜在的缓存过期问题，并至少确保断点包含系统提示的结尾。

此外，如果你正在使用像 vLLM 这样的框架自托管模型，请确保启用了前缀/提示缓存，并且你正在使用会话 ID 等技术在分布式工作节点之间一致地路由请求。

遮蔽，而非移除

随着代理能力的增强，其行动空间自然变得更加复杂——简单来说，工具数量爆炸式增长。最近流行的MCP只会火上浇油。如果你允许用户自定义工具，相信我：总会有人将数百个神秘工具插入到你精心策划的行动空间中。结果，模型更可能选择错误的行动或采取低效的路径。简而言之，你武装过度的代理变得更加愚蠢。
一个自然的反应是设计一个动态行动空间——可能是使用类似于RAG的方法按需加载工具。我们在Manus中也尝试过这种方法。但我们的实验表明了一个明确的规则：除非绝对必要，避免在迭代过程中动态添加或移除工具。这主要有两个原因：
1.在大多数LLM中，工具定义在序列化后位于上下文的前部，通常在系统提示之前或之后。因此任何更改都会使后续所有动作和观察的KV缓存失效。

2.当先前的动作和观察仍然引用当前上下文中不再定义的工具时，模型会感到困惑。如果没有约束解码，这通常会导致模式违规或幻觉动作。
为了解决这个问题并仍然改进动作选择，Manus使用上下文感知的状态机来管理工具可用性。它不是移除工具，而是在解码过程中掩蔽token的logits，以基于当前上下文阻止（或强制）选择某些动作。



在实践中，大多数模型提供商和推理框架支持某种形式的响应预填充，这允许你在不修改工具定义的情况下约束动作空间。函数调用通常有三种模式（我们将使用 NousResearch

的 Hermes 格式 作为示例）：
•自动 – 模型可以选择调用或不调用函数。通过仅预填充回复前缀实现：
<|im_start|>assistant

•必需 – 模型必须调用函数，但选择不受约束。通过预填充到工具调用令牌实现：
<|im_start|>assistant<tool_call>

•指定 – 模型必须从特定子集中调用函数。通过预填充到函数名称的开头实现：
<|im_start|>assistant<tool_call>{"name": "browser_通过这种方式，我们通过直接掩码token的logits来约束动作选择。例如，当用户提供新输入时，Manus必须立即回复而不是执行动作。我们还有意设计了具有一致前缀的动作名称——例如，所有与浏览器相关的工具都以browser_开头，命令行工具以shell_开头。这使我们能够轻松确保代理在给定状态下只从特定工具组中进行选择而无需使用有状态的logits处理器。

这些设计有助于确保Manus代理循环保持稳定——即使在模型驱动的架构下。

使用文件系统作为上下文

现代前沿LLM现在提供128K令牌或更多的上下文窗口。但在真实世界的代理场景中，这通常不够，有时甚至是一种负担。有三个常见的痛点：

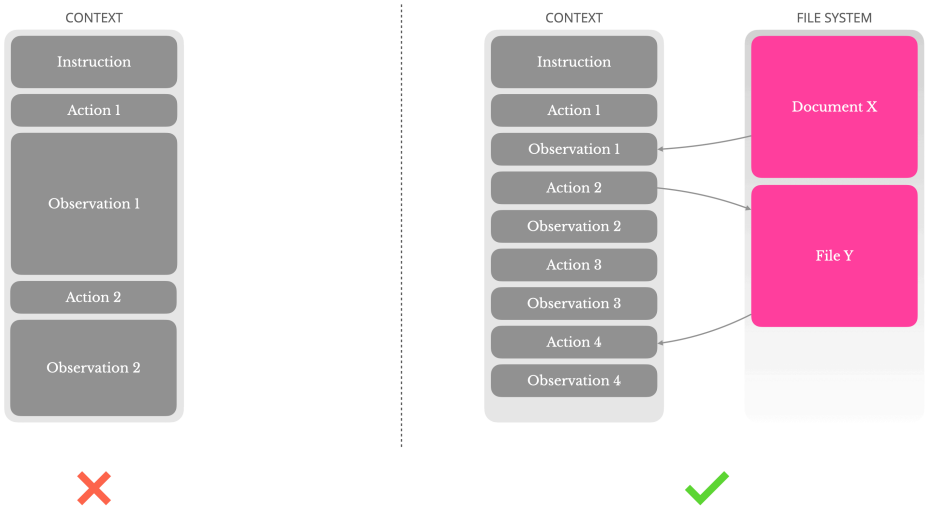
1.观察结果可能非常庞大，尤其是当代理与网页或PDF等非结构化数据交互时。很容易超出上下文限制。

2.模型性能往往会下降，超过一定的上下文长度后，即使技术上支持该窗口大小。

3.长输入成本高昂，即使使用前缀缓存。你仍然需要为传输和预填充每个token付费。

为了解决这个问题，许多代理系统实现了上下文截断或压缩策略。但过度激进的压缩不可避免地导致信息丢失。这个问题是根本性的：代理本质上必须根据所有先前状态预测下一个动作——而你无法可靠地预测哪个观察结果可能在十步之后变得至关重要。从逻辑角度看，任何不可逆的压缩都带有风险。这就是为什么我们在Manus中将文件系统视为终极上下文：大小不受限制，天然持久化，并且代理可以直接操作。模型学会按需写入和读取文件——不仅将文件系统用作存储，还用作结构化的外部记忆。

Use the File System as Context



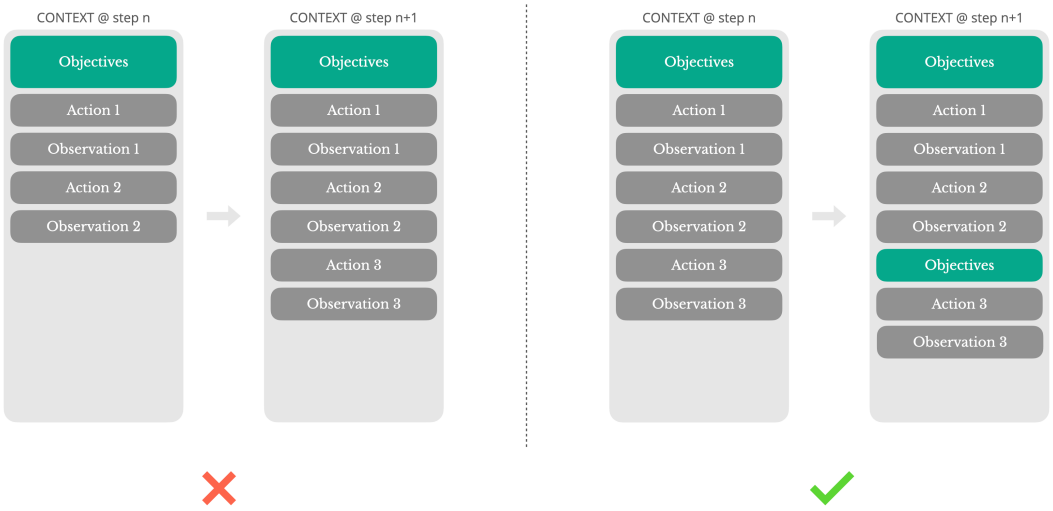
我们的压缩策略始终设计为可恢复的。例如，只要保留URL，网页内容就可以从上下文中移除；如果沙盒中仍然保留文档路径，则可以省略文档内容。这使得Manus能够缩短上下文长度，而不会永久丢失信息。在开发这个功能时，我发现自己在想象状态空间模型(State Space Model, SSM)在智能体环境中有效工作需要什么条件。与Transformer不同，SSM缺乏完整的注意力机制，并且在处理长距离的后向依赖关系时表现不佳。但如果它们能够掌握基于文件的记忆——将长期状态外部化而不是保存在上下文中——那么它们的速度和效率可能会开启一类新型智能体。基于SSM的智能体可能是神经图灵机真正的继任者。

通过复述操控注意力

如果你使用过Manus，你可能注意到一个有趣的现象：在处理复杂任务时，它倾向于创建一个todo.md文件——并在任务进行过程中逐步更新它，勾选已完成的项目。

这不仅仅是可爱的行为——这是一种操控注意力的刻意机制。

Manipulate Attention Through Recitation



Manus中的一个典型任务平均需要大约50次工具调用。这是一个很长的循环——由于Manus依赖LLM进行决策，它很容易偏离主题或忘记早期目标，尤其是在长上下文或复杂任务中。

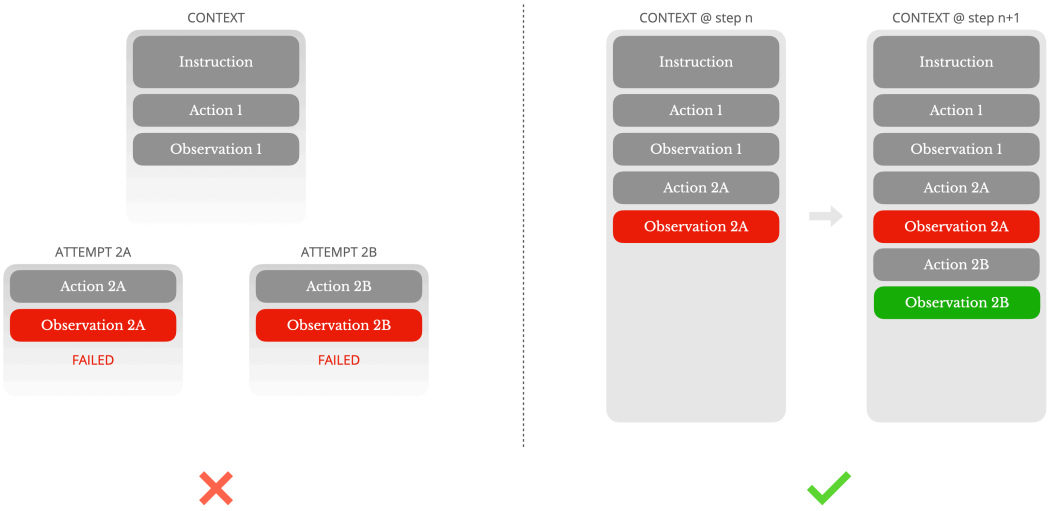
通过不断重写待办事项列表，Manus将其目标复述到上下文的末尾。这将全局计划推入模型的近期注意力范围内，避免了"丢失在中间"的问题，并减少了目标不一致。实际上，它使用自然语言来使自己的注意力偏向任务目标——而不需要特殊的架构变更。

保留错误的内容

代理会犯错。这不是bug——这是现实。语言模型会产生幻觉，环境会返回错误，外部工具会出现异常行为，意外的边缘情况随时都会出现。在多步骤任务中，失败不是例外；它是循环的一部分。

然而，一个常见的冲动是隐藏这些错误：清理痕迹，重试操作，或重置模型的状态并将其留给神奇的"温度"。这感觉更安全，更受控制。但这是有代价的：擦除失败会移除证据。没有证据，模型就无法适应。

Keep the Wrong Stuff In



根据我们的经验，改善代理行为最有效的方法之一出奇地简单：将错误的尝试保留在上下文中。当模型看到一个失败的行动——以及由此产生的观察结果或堆栈跟踪——它会隐式地更新其内部信念。这会改变其先验，降低重复相同错误的可能性。事实上，我们认为错误恢复是真正代理行为的最明显指标之一。然而，在大多数学术工作和公共基准测试中，这一点仍然代表性不足，它们通常关注理想条件下的任务成功。

不要被少样本示例所困

少样本提示是提高LLM输出的常用技术。但在代理系统中，它可能会以微妙的方式适得其反。语言模型是优秀的模仿者；它们模仿上下文中的行为模式。如果你的上下文充满了类似的过去行动-观察对，模型将倾向于遵循该模式，即使这不再是最优的。
这在涉及重复决策或行动的任务中可能很危险。例如，当使用Manus帮助审查20份简历时，代理通常会陷入一种节奏——仅仅因为这是它在上下文中看到的，就重复类似的行动。这导致偏离、过度泛化，或有时产生幻觉。

Don't Get Few-Shotted

解决方法是增加多样性。Manus在行动和观察中引入少量的结构化变化——不同的序列化模板、替代性措辞、顺序或格式上的微小噪音。这种受控的随机性有助于打破模式并调整模型的注意力。换句话说，不要让自己陷入少样本学习的窠臼。你的上下文越单一，你的智能体就变得越脆弱。

结论

上下文工程仍然是一门新兴的科学——但对于智能体系统来说，它已经是必不可少的。模型可能变得更强大、更快速、更经济，但再多的原始能力也无法替代对记忆、环境和反馈的需求。你如何塑造上下文最终决定了你的智能体的行为方式：它运行的速度、恢复的效果以及扩展的范围。

在Manus，我们通过反复的重写、死胡同以及面向数百万用户的实际测试学到了这些经验。我们在这里分享的内容并非放之四海而皆准的真理——但这些是对我们有效的模式。如果它们能帮助你避免哪怕一次痛苦的迭代，那么这篇文章就达到了它的目的。

智能体的未来将一次构建一个上下文。好好设计它们吧。